STEP 1 — Install & Import Libraries

```
!pip install nltk scikit-learn pandas numpy


import numpy as np
import pandas as pd
import nltk
import string

from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer


nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

STEP 2 — Create Dataset (15 documents)

```
documents = [
    "Machine learning is a part of artificial intelligence.",
    "Machine learning is a subset of AI.",
    "Deep learning is a branch of machine learning.",
    "Artificial intelligence is transforming the world.",
    "AI is changing industries rapidly.",
    "The cat is sitting on the mat.",
    "A cat sits on a mat.",
    "Dogs are playing in the park.",
    "A group of dogs play outside.",
    "The weather is sunny today.",
    "It is a bright sunny day.",
    "I love programming in Python.",
    "Coding in Python is enjoyable.",
    "Football is a popular sport.",
    "Cricket is widely played in India."
]
```

STEP 3 — Preprocessing Function

```
nltk.download('punkt_tab')
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
```

```
    text = text.lower()


    text = text.translate(str.maketrans('', '', string.punctuation))


    words = word_tokenize(text)


    words = [lemmatizer.lemmatize(word) for word in words if word not in stop_words]

    return " ".join(words)

processed_docs = [preprocess(doc) for doc in documents]

print(processed_docs[:5])
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
['machine learning part artificial intelligence', 'machine learning subset ai', 'deep learning branch machine learning', 'artifi
```

STEP 4 — Cosine Similarity (TF-IDF)

```
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(processed_docs)

cos_sim = cosine_similarity(tfidf_matrix)


cos_df = pd.DataFrame(cos_sim)
print(cos_df)
```

```
          0         1         2         3         4         5         6   \
0   1.000000  0.362769  0.418223  0.418066  0.000000  0.000000  0.000000
1   0.362769  1.000000  0.468552  0.000000  0.226385  0.000000  0.000000
2   0.418223  0.468552  1.000000  0.000000  0.000000  0.000000  0.000000
3   0.418066  0.000000  0.000000  1.000000  0.000000  0.000000  0.000000
4   0.000000  0.226385  0.000000  0.000000  1.000000  0.000000  0.000000
5   0.000000  0.000000  0.000000  0.000000  0.000000  1.000000  0.601276
6   0.000000  0.000000  0.000000  0.000000  0.000000  0.601276  1.000000
7   0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
8   0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
9   0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
10  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
11  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
12  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
13  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
14  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000

        7       8         9         10        11        12   13   14
0   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
1   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
2   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
3   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
4   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
5   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
6   0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
7   1.0000  0.2345  0.000000  0.000000  0.000000  0.000000  0.0  0.0
8   0.2345  1.0000  0.000000  0.000000  0.000000  0.000000  0.0  0.0
9   0.0000  0.0000  1.000000  0.273784  0.000000  0.000000  0.0  0.0
10  0.0000  0.0000  0.273784  1.000000  0.000000  0.000000  0.0  0.0
11  0.0000  0.0000  0.000000  0.000000  1.000000  0.273784  0.0  0.0
12  0.0000  0.0000  0.000000  0.000000  0.273784  1.000000  0.0  0.0
13  0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  1.0  0.0
14  0.0000  0.0000  0.000000  0.000000  0.000000  0.000000  0.0  1.0
```

```
pairs = []

for i in range(len(cos_sim)):
    for j in range(i+1, len(cos_sim)):
        pairs.append((i, j, cos_sim[i][j]))


pairs = sorted(pairs, key=lambda x: x[2], reverse=True)

print("Top 5 Cosine Similar Pairs:\n")
for i, j, score in pairs[:5]:
    print(f"Doc {i} & Doc {j} → Score: {score:.3f}")
```

```
Top 5 Cosine Similar Pairs:

Doc 5 & Doc 6 → Score: 0.601
Doc 1 & Doc 2 → Score: 0.469
Doc 0 & Doc 2 → Score: 0.418
Doc 0 & Doc 3 → Score: 0.418
Doc 0 & Doc 1 → Score: 0.363
```

STEP 5 — Jaccard Similarity

```
def jaccard_similarity(doc1, doc2):
    set1 = set(doc1.split())
    set2 = set(doc2.split())

    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))

    return intersection / union

jac_matrix = np.zeros((len(processed_docs), len(processed_docs)))

for i in range(len(processed_docs)):
    for j in range(len(processed_docs)):
        jac_matrix[i][j] = jaccard_similarity(processed_docs[i], processed_docs[j])

jac_df = pd.DataFrame(jac_matrix)
print(jac_df)
```

```
          0         1         2         3         4    5    6         7   \
0   1.000000  0.285714  0.285714  0.285714  0.000000  0.0  0.0  0.000000
1   0.285714  1.000000  0.333333  0.000000  0.142857  0.0  0.0  0.000000
2   0.285714  0.333333  1.000000  0.000000  0.000000  0.0  0.0  0.000000
3   0.285714  0.000000  0.000000  1.000000  0.000000  0.0  0.0  0.000000
4   0.000000  0.142857  0.000000  0.000000  1.000000  0.0  0.0  0.000000
5   0.000000  0.000000  0.000000  0.000000  0.000000  1.0  0.5  0.000000
6   0.000000  0.000000  0.000000  0.000000  0.000000  0.5  1.0  0.000000
7   0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  1.000000
8   0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.166667
9   0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.000000
10  0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.000000
11  0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.000000
12  0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.000000
13  0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.000000
14  0.000000  0.000000  0.000000  0.000000  0.000000  0.0  0.0  0.000000

          8    9    10   11   12   13   14
0   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
1   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
2   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
3   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
4   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
5   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
6   0.000000  0.0  0.0  0.0  0.0  0.0  0.0
7   0.166667  0.0  0.0  0.0  0.0  0.0  0.0
8   1.000000  0.0  0.0  0.0  0.0  0.0  0.0
9   0.000000  1.0  0.2  0.0  0.0  0.0  0.0
10  0.000000  0.2  1.0  0.0  0.0  0.0  0.0
11  0.000000  0.0  0.0  1.0  0.2  0.0  0.0
12  0.000000  0.0  0.0  0.2  1.0  0.0  0.0
13  0.000000  0.0  0.0  0.0  0.0  1.0  0.0
14  0.000000  0.0  0.0  0.0  0.0  0.0  1.0
```

STEP 6 — WordNet Semantic Similarity

```
from nltk.corpus import wordnet as wn

def wordnet_similarity(word1, word2):
    syns1 = wn.synsets(word1)
    syns2 = wn.synsets(word2)

    max_score = 0

    for s1 in syns1:
        for s2 in syns2:
            score = s1.wup_similarity(s2)
            if score is not None:
                max_score = max(max_score, score)

    return max_score


def sentence_similarity(sent1, sent2):
    words1 = sent1.split()
    words2 = sent2.split()

    scores = []

    for w1 in words1:
        max_sim = 0
        for w2 in words2:
            sim = wordnet_similarity(w1, w2)
            if sim:
                max_sim = max(max_sim, sim)
        scores.append(max_sim)

    return sum(scores)/len(scores) if scores else 0


print("WordNet Similarity:\n")
for i in range(10):
    sim = sentence_similarity(processed_docs[i], processed_docs[i+1])
    print(f"Doc {i} & Doc {i+1} → {sim:.3f}")
```

```
WordNet Similarity:

Doc 0 & Doc 1 → 0.798
Doc 1 & Doc 2 → 0.794
Doc 2 & Doc 3 → 0.702
Doc 3 & Doc 4 → 0.621
Doc 4 & Doc 5 → 0.620
Doc 5 & Doc 6 → 1.000
Doc 6 & Doc 7 → 0.730
Doc 7 & Doc 8 → 0.912
Doc 8 & Doc 9 → 0.469
Doc 9 & Doc 10 → 0.803
```